

Dokumentation

Modul 335 Mobile Applikation realisieren

BookCrux

Dateiname: Uek335_Dokumentation_Lisa_Tristan.docx

Speicherdatum: Wednesday, May 27, 2026

Version: 1.0

Autor: Tristan Ackermann, Yelyzaveta Pohorielova

Team: 2

Betrieb: Noseryoung

Table of Contents

1 Schriftliche Dokumente	3
1.1 Anforderungen.....	3
1.2 Architektur / Design	12
1.3 Technischer Bericht.....	13
1.3.1 Testing.....	14
1.3.2 Zukünftige Features	18
1.4 Installationsanleitung	<i>Error! Bookmark not defined.</i>
1.5 Code Dokumentation	18
1.6 Aufgabenaufteilung.....	19

1 Schriftliche Dokumente

1.1 Anforderungen

Funktionale Anforderungen

User-Story 1

A	Mit bestehendem Konto anmelden	3
Ich als: registrierter Nutzer Möchte: mich mit E-Mail und Passwort anmelden Um: auf meine persönlichen Daten zuzugreifen		
Die User-Story ist fertig wenn: • bei korrekten Daten der Token gespeichert wird und ich zur Hauptseite weitergeleitet werde • bei falschen Daten eine verständliche Fehlermeldung erscheint.		

User-Story 2

A	Neues Konto erstellen	3
Ich als: neuer Nutzer Möchte: mich mit Vornamen, Nachnamen, E-Mail, Passwort und Alter registrieren Um: Zugang zur App zu erhalten		
Die User-Story ist Fertig wenn: • das Formular validiert wird • der Account im Backend erstellt wird • der Token im SecureStore gespeichert ist		

User-Story 3

B	Abmelden	1
Ich als: angemeldeter Nutzer Möchte: mich abmelden können Um: mein Konto auf geteilten Geräten zu schützen		
Die User-Story ist fertig, wenn: • der Token aus dem SecureStore gelöscht wird • ich zur Login-Seite weitergeleitet werde		

User-Story 4

B	Eigenes Profil einsehen	1
Ich als: angemeldeter Nutzer Möchte: meine Profildaten (Name, E-Mail, Alter) einsehen können Um: meine gespeicherten Informationen zu überprüfen		
Die User-Story ist fertig, wenn: • die Profildaten vom Backend geladen und korrekt auf der Profilseite angezeigt werden		

User-Story 5

B	Account permanent löschen	1
Ich als: Nutzer Möchte: mein Konto löschen können Um: meine Daten vollständig aus dem System zu entfernen		
Die User-Story ist fertig, wenn: • ein Bestätigungsdialog erscheint, das Konto im Backend gelöscht wird • der Token entfernt wird • ich zur Registrierungsseite weitergeleitet werde		

User-Story 6

A	Bibliothek durchsuchen und filtern	5
Ich als: Nutzer Möchte: Bücher nach Titel suchen und nach Seitenanzahl filtern können Um: schnell das passende Buch zu finden		
Die User-Story ist fertig, wenn: • die Suche funktioniert • der Slider die Ergebnisse filtert • eine leere Liste mit einer verständlichen Meldung angezeigt wird, falls es keine Ergebnisse gibt		

User-Story 7

B	Buch erstellen	2
Ich als: angemeldeter Nutzer Möchte: ein neues Buch erfassen Um: die Bücherliste zu erweitern		
Die User-Story ist fertig, wenn: • wenn ein Formular mit allen Buchfelder vorhanden ist • Die Eingabe validiert wird • Das neue Buch in der Liste erscheint		

User-Story 8

B	Buch bearbeiten	2
Ich als: angemeldeter Nutzer Möchte: ein bestehendes Buch bearbeiten Um: fehlerhafte oder veraltete Angaben zu korrigieren		
Die User-Story ist fertig, wenn: • Die bestehenden Daten im Formular vorausgefüllt sind • Änderungen gespeichert und in der Liste aktualisiert werden		

User-Story 9

B	Buch aus der Bibliothek entfernen	1
Ich als: Nutzer Möchte: ein Buch aus der Bibliothek löschen können Um: die Liste aktuell und übersichtlich zu halten		
Die User-Story ist fertig, wenn: • ein Bestätigungsdialog erscheint, das Buch nach Bestätigung im Backend gelöscht wird • die Liste sich sofort aktualisiert		

User-Story 10

B	Navigationsleiste	1
Ich als: angemeldeter Nutzer Möchte: über eine Navigationsleiste zwischen den Seiten wechseln Um: schnell und einfach durch die App navigieren zu können		
Die User-Story ist fertig, wenn: • ein Navigationsbar nach dem anmelden sichtbar sein • Alle taps wie z.B Bücher und Profil vorhanden • Der Aktive Tap visuell hervorgehoben wird • Die Navigation nur für eingeloggte User sichtbar ist		

1.2 Architektur / Design

Anforderungen an das Endprodukt

Die App Book Details Checker wurde im Rahmen des Moduls 335 für die Firma BookCruxLibrary entwickelt. Ziel war es, eine mobile Applikation zu realisieren, mit welcher Benutzer Bücher verwalten und deren Details einsehen können. Die Umsetzung erfolgte mit React Native in Verbindung mit React Native Paper für die UI-Komponenten sowie SecureStorage für die sichere Speicherung von Benutzerdaten. Als Entwicklungsumgebung wurden Visual Studio Code und Android Studio (AVD) eingesetzt.

Die Anforderungen wurden in drei Kategorien eingeteilt:

MUSS-Anforderungen (zwingend erfüllt):

- Register-Screen mit E-Mail- und Passwort-Validierung

- Login-Screen mit Fehlermeldung bei ungültigen Eingaben
- Navigation zwischen Bücher-Screen und Profil-Screen nach erfolgreichem Login
- Vollständige CRUD-Funktionalität auf dem Bücher-Screen (Erstellen, Anzeigen, Bearbeiten, Löschen)
- Filterfunktion nach Buchname
- Logout-Funktion mit Weiterleitung auf den Login-Screen

SOLL-Anforderungen (umgesetzt sofern Zeit vorhanden):

- Zusätzlicher Home-Screen als Übersicht nach dem Login
- Erweiterte Filteroptionen für Bücher
- Zusätzliche Felder bei der Registrierung (Vor-/Nachname, Alter)
- Anzeige dieser Zusatzangaben im Profil-Screen

KANN-Anforderungen (optionale Erweiterungen):

- Weiterleitung auf den Home-Screen nach Login statt auf den Bücher-Screen
- Account-Löschfunktion

Struktur und Verhalten des Codes

Die App ist nach dem komponentenbasierten Prinzip von React Native aufgebaut. Die Navigation zwischen den Screens wird über einen Stack Navigator (Pre-Login) sowie einen Bottom Tab Navigator (Post-Login) realisiert. Die Screens gliedern sich wie folgt:

- **RegisterScreen** Eingabe und Validierung von E-Mail, Passwort und optionalen Profilangaben
- **LoginScreen** Authentifizierung des Benutzers, Fehlermeldung bei falschen Daten
- **HomeScreen** (*SOLL*) Übersichtsseite nach dem Login
- **BooksScreen** Liste aller Bücher mit Suchfeld, Create/Update/Delete-Aktionen
- **BookDetailScreen** Detailansicht eines einzelnen Buches
- **ProfileScreen** Anzeige der Nutzerdaten, Logout-Button

Benutzerdaten werden mit SecureStorage lokal und verschlüsselt gespeichert. Die Bücherdaten werden im lokalen State verwaltet bzw. persistent gespeichert.

1.3 Technischer Bericht

Hier ist Das Testing und zukünftige Features Dokumentiert

1.3.1 Testing

Test 1 Happy Path

Test	Search
Component	SearchBar
Priority	High
Discription	User sucht ein existierendes Buch
Pre-requisites	Applikation ist geöffnet und Docker Container läuft. User muss angemeldet sein
Test Steps	1. Navigiere mit der Navigation bar zu der Search Seite. 2. Gib ein beliebiger buch Name
Experte Resultat	Falls ein Buch mit dem Namen existiert, sollte ein Buch erscheinen
Actual Resultat	Das Buch erscheint
Status	OK
Test Executet By	Tristan Ackermann

Test 1 Error Path 1

Test	Search
Component	SearchBar
Priority	High
Discription	User sucht ein Buch, das nicht existiert
Pre-requisites	Applikation ist geöffnet und Docker Container läuft. User muss angemeldet sein
Test Steps	1. Navigiere mit der Navigation bar zu der Search Seite. 2. Gib ein Buch, das nicht existiert
Experte Resultat	Kin Buch Sollte Erscheinen
Actual Resultat	Kein Buch erscheint
Status	OK
Test Executet By	Tristan Ackermann

Test 2 Happy Path

Test	Registrierung
Component	Register Page
Priority	High
Discription	Nutzer registriert sich mit gültigen Daten
Pre-requisites	App läuft, kein bestehendes Konto
Test Steps	1. Registrierungsseite öffnen 2. Alle Felder korrekt ausfüllen 3. SIGN UP tippen
Experte Resultat	Nutzer wird zur Hauptseite weitergeleitet, Token im SecureStore gespeichert
Actual Resultat	Nutzer wird zur Hauptseite weitergeleitet, Token im SecureStore gespeichert
Status	OK
Test Executet By	Lisa

Test 2 Error Path 1

Test	Registrierung
Component	Register Page
Priority	High
Discription	Nutzer gibt ungültiges E-Mail-Format ein
Pre-requisites	App läuft, kein bestehendes Konto
Test Steps	1. Registrierungsseite öffnen 2. "ungueltigeemail" als E-Mail eingeben 3. SIGN UP tippen
Experte Resultat	Fehlermeldung "Invalid email" unter dem Feld angezeigt
Actual Resultat	Fehlermeldung "Invalid email" unter dem Feld angezeigt
Status	OK
Test Executet By	Lisa

Test 2 Error Path 2

Test	Registrierung
Component	Register Page
Priority	High
Discription	Nutzer gibt Passwort mit weniger als 6 Zeichen ein
Pre-requisites	App läuft, kein bestehendes Konto
Test Steps	1. Registrierungsseite öffnen 2. Passwort "abc" eingeben 3. SIGN UP tippen
Experte Resultat	Fehlermeldung "Min. 6 characters" unter dem Feld angezeigt
Actual Resultat	Fehlermeldung "Min. 6 characters" unter dem Feld angezeigt
Status	OK
Test Executet By	Lisa

Test 3 Happy Path

Test	Erfolgreicher Login
Component	LoginPage
Priority	High
Discription	Nutzer meldet sich mit korrekten Daten an
Pre-requisites	Bestehendes Konto im Backend
Test Steps	1. Login-Seite öffnen 2. Gültige E-Mail & Passwort eingeben 3. SIGN IN tippen
Experte Resultat	Weiterleitung zur Hauptseite, Token gespeichert
Actual Resultat	Weiterleitung zur Hauptseite, Token gespeichert
Status	OK
Test Executet By	Tristan Ackermann

Test 3 Error Path 1

Test	Unbekannte E-Mail
Component	LoginPage
Priority	High
Discription	Nutzer gibt nicht registrierte E-Mail ein
Pre-requisites	Bestehendes Konto im Backend
Test Steps	1. Login-Seite öffnen 2. Unbekannte E-Mail eingeben 3. SIGN IN tippen
Experte Resultat	Fehlermeldung "Incorrect E-Mail or password" angezeigt
Actual Resultat	Fehlermeldung "Incorrect E-Mail or password" angezeigt
Status	OK
Test Executet By	Tristan Ackermann

Test 3 Error Path 2

Test	Leere Email
Component	LoginPage
Priority	High
Discription	Nutzer gibt keine Email ein
Pre-requisites	Bestehendes Konto im Backend
Test Steps	1. Login-Seite öffnen 2. Gültiges Passowrd, kein E-Mail eingeben 3. SIGN IN tippen
Experte Resultat	Fehlermeldung "Invalid E-Mail" angezeigt
Actual Resultat	Fehlermeldung "Invalid E-Mail" angezeigt
Status	OK
Test Executet By	Tristan Ackermann

Test 4 Happy Path

Test	Buch erstellt, bearbeitet und gelöscht
Component	BooksPageTmpl, BookModal
Priority	High
Discription	Vollständiger CRUD-Zyklus funktioniert korrekt
Pre-requisites	Nutzer eingeloggt
Test Steps	2. Formular ausfüllen 3. Erstellen tippen 4. Bearbeiten-Icon tippen 5. Titel ändern 6. Speichern tippen 7. Löschen-Icon tippen 8. Bestätigen
Experte Resultat	Fehlermeldung "Invalid E-Mail" angezeigt

Actual Resultat	Fehlermeldung "Invalid E-Mail" angezeigt
Status	OK
Test Executed By	Lisa

Test 4 Error Path 1

Test	Erstellen eines Buches ohne Titel
Component	BookModal
Priority	High
Description	Nutzer sendet Formular ohne Titel ab
Pre-requisites	Nutzer eingeloggt
Test Steps	1. Login-Seite öffnen 2. Gültiges Passwort, kein E-Mail eingeben 3. SIGN IN tippen
Expected Resultat	Fehlermeldung "Invalid E-Mail" angezeigt
Actual Resultat	Fehlermeldung "Invalid E-Mail" angezeigt
Status	OK
Test Executed By	Lisa

Test 5 Happy Path

Test	Erfolgreicher Logout
Component	ProfilePage
Priority	High
Description	Nutzer meldet sich erfolgreich ab
Pre-requisites	Nutzer eingeloggt
Test Steps	1. Profilseite öffnen 2. LOGOUT tippen
Expected Resultat	Token gelöscht, Weiterleitung zur Login-Seite
Actual Resultat	Token gelöscht, Weiterleitung zur Login-Seite
Status	OK
Test Executed By	Tristan Ackermann

Test 6 Happy Path

Test	Profil wird korrekt geladen
Component	ProfilePage
Priority	High
Description	Profil Daten werden geladen und angezeigt
Pre-requisites	Nutzer eingeloggt, Daten im Backend
Test Steps	1. Einloggen 2. Profilseite öffnen

Experte Resultat	Vorname, Nachname, E-Mail und Alter korrekt angezeigt
Actual Resultat	Vorname, Nachname, E-Mail und Alter korrekt angezeigt
Status	OK
Test Executed By	Lisa

Test 7 Happy Path

Test	Konto erfolgreich gelöscht
Component	ProfilePage
Priority	High
Description	Nutzer bestätigt Löschung, Konto wird entfernt
Pre-requisites	Nutzer eingeloggt
Test Steps	1. Profilseite öffnen 2. DELETE tippen 3. Löschen im Dialog bestätigen
Experte Resultat	Konto gelöscht, Token gelöscht, Weiterleitung zur Registrierung
Actual Resultat	Konto gelöscht, Token gelöscht, Weiterleitung zur Registrierung
Status	OK
Test Executed By	Lisa

1.3.2 Zukünftige Features

In zukünftigen Versionen der App sind folgende Erweiterungen geplant:

Mit dem Leseregal können Nutzer ihre Bücher als gelesen, am Lesen oder zum Lesen markieren. So verwalten sie ihre eigene Bibliothek. Es gibt ein System, mit dem Nutzer Bücher bewerten und Kommentare schreiben können.

Es ist eine Profilbearbeitung geplant. Nutzer können dann zum Beispiel ihren Namen, ihre E-Mail-Adresse und ihr Alter selbst anpassen. Es soll eine Funktion geben, mit der man das Passwort per E-Mail zurücksetzen kann. So werden die Konten sicherer. Ausserdem kann man ein Profilbild hochladen. Und man kann Bilder für das Deckblatt vom lokalen Speicher hochladen.

1.4 Code Dokumentation

Neben den Dokumenten soll im TSDocs im Code verwendet werden. TSDocs ist für TypeScript

das, was JavaDocs für Java ist. Es beschreibt die Funktionalitäten des Codes und lässt sich mit zusätzlichen Tools extrahieren und zu einer Dokumentation zusammengefügt. TSDocs muss für die wichtigsten Funktionen vorhanden sein.

1.5 Aufgabenaufteilung

Aufgabe	Verantwortlich
Mockups & Mockup Responsiveness	Tristan & Lisa
Login & Register Pages + Logik	Lisa
Books Search Page + Search Logik	Tristan
Add Book Page + Logik	Tristan
Info Page	Lisa
Menu Bar + Navigation	Tristan
Edit & Delete Book Modal + Logik	Lisa
Projektdokumentation	Tristan & Lisa